

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: G.Sravva

Roll No.: A24126552257

Date: 19-09-2026

TASK 2 — NODE.JS FILE MANAGEMENT SYSTEM (CO-2)

1. TITLE

Develop a Server-Side Node.js File Management Application Using the File System (fs) and readline Modules

2. AIM

To construct an interactive command-line Node.js application that accepts a target filename and text content from the user, and sequentially executes file operations: creating and writing to the file, reading its contents, appending additional data, and displaying the final contents using the built-in fs and readline modules.

3. OBJECTIVE

The objectives of this task are:

- To configure and execute server-side JavaScript applications in the Node.js runtime environment.
- To utilize Node.js built-in modules (fs, path, and readline) without relying on third-party libraries.
- To capture interactive user terminal inputs sequentially using the readline interface.
- To write data to a file using fs.writeFile / fs.writeFileSync.
- To read and decode textual data from a file using fs.readFile with the 'utf-8' character encoding.
- To append content to an existing file using fs.appendFile / fs.appendFileSync.
- To implement error handling using try...catch blocks and asynchronous callbacks.

4. THEORY

Node.js provides a server-side JavaScript runtime built on Google Chrome's V8 JavaScript engine. It employs an event-driven, single-threaded, non-blocking I/O model suitable for data-intensive real-time applications.

Key Concepts Used:

1. The File System (fs) Module:

- Node.js provides the built-in fs module to access and manipulate physical files on the host operating system.

- It offers both synchronous (*Sync) and asynchronous (callback or promise-based) methods.
2. **Core File Operations:**
 - `fs.writeFile(path, data, [options], callback)`: Asynchronously writes data to a file, replacing the file if it already exists.
 - `fs.readFile(path, [options], callback)`: Asynchronously reads the entire contents of a file. Specifying 'utf-8' decodes the raw byte buffer into a readable string.
 - `fs.appendFile(path, data, [options], callback)`: Asynchronously appends data to a file, creating the file if it does not already exist.
 - `fs.stat(path, callback)`: Inspects file metadata, including file size in bytes, creation time, and last modification timestamp.
 3. **The readline Module:**
 - Provides an interface for reading streams of data line-by-line from readable streams, such as `process.stdin`.
 - Methods: `readline.createInterface()`, `rl.question(prompt, callback)`, and `rl.close()`.
 4. **Buffer vs String Encoding:**
 - When reading files without specifying an encoding format, Node.js returns a raw binary Buffer. Supplying 'utf-8' decodes bytes into UTF-8 characters.
 5. **Exception Handling:**
 - Server-side file I/O operations can encounter OS-level errors (insufficient permissions `EACCES`, non-existent paths `ENOENT`, or disk exhaustion). Wrapping operations in `try...catch` or checking the `err` argument in callbacks ensures application stability.

5. ALGORITHM / PROCEDURE

1. Import built-in modules: `const fs = require('fs'), const readline = require('readline'), and const path = require('path')`.
2. Configure a `readline.Interface` instance passing `process.stdin` as the input stream and `process.stdout` as the output stream.
3. Define an asynchronous or promise-based wrapper for `rl.question()` to enable readable sequential execution.
4. **Step 1 (Input Filename):** Prompt the user to enter a filename. Sanitize the input and verify that a valid .txt or equivalent extension is present.
5. **Step 2 (Create & Write):** Prompt the user to enter initial content. Execute `fs.writeFileSync()` or `fs.writeFile()` to create the file and write the text to disk.
6. **Step 3 (Read & Verify):** Read the file back from disk using `fs.readFileSync(filepath, 'utf-8')`. Output the contents to the console along with confirmation logs.
7. **Step 4 (Append Data):** Prompt the user for additional text content to append. Execute `fs.appendFileSync()` or `fs.appendFile()` to append the text without overwriting existing data.
8. **Step 5 (Display Final Contents):** Read the updated file, display the concatenated contents, and print file statistics (`fs.statSync()` size and timestamp).
9. Close the readline interface using `rl.close()` and terminate execution cleanly.
10. Verify program execution through the terminal.

6. PROGRAM

fileManager.js

```
/** Node.js File Management Application
 * Demonstrates: File Creation, Writing, Reading, Appending, and Verification */
const fs = require('fs');
const readline = require('readline');
const path = require('path');
// 1. Initialize readline interface for standard input and output
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});
// 2. Promisify readline question for clean sequential async/await flow
const askQuestion = (query) => {
  return new Promise((resolve) => rl.question(query, resolve));
};
// 3. Main File Management Controller Function
async function runFileManager() {

  console.log("\n=====
  =====");
  console.log('          NODE.JS FILE SYSTEM MANAGEMENT APPLICATION          ');

  console.log('=====
  =====');
  try {
    // STEP 1: Prompt user for Filename
    let fileName = await askQuestion("\n[1] Enter the filename to create (e.g.,
student_record.txt): ");
    fileName = fileName.trim();
    if (!fileName) {
      fileName = 'default_record.txt';
      console.log(`>> No filename entered. Defaulting to: ${fileName}`);
    }
    // Ensure file is saved in current working directory
    const filePath = path.join(__dirname, fileName);
    // STEP 2: Prompt user for Initial Content & Write to File
    const initialContent = await askQuestion("\n[2] Enter initial content to write to the
file:\n> ");
    console.log(`\n--> Creating and writing to "${fileName}"...`);
    fs.writeFileSync(filePath, initialContent + '\n', 'utf-8');
    console.log(`[✓] File "${fileName}" created and written successfully!`);
    // STEP 3: Read Initial Contents from File
    console.log(`\n--> Reading contents from file...`);
    const readContent1 = fs.readFileSync(filePath, 'utf-8');
    console.log('-----');
    console.log('FILE CONTENTS (AFTER INITIAL WRITE):');
    console.log(readContent1.trim());
```

```

        console.log('-----');
        // STEP 4: Prompt user for Additional Content to Append
        const appendContent = await askQuestion('\n[3] Enter additional content to append to
the file:\n> ');
        console.log('\n--> Appending content to "${fileName}"...`);
        fs.appendFileSync(filePath, appendContent + '\n', 'utf-8');
        console.log(`[✓] Additional content appended successfully!`);
        // STEP 5: Display Final File Contents and Metadata
        console.log('\n--> Reading final updated file contents...');
        const finalContent = fs.readFileSync(filePath, 'utf-8');
        const fileStats = fs.statSync(filePath);

console.log('=====
====');
        console.log(`FINAL FILE CONTENTS (${fileName}):`);

console.log('=====
====');
        console.log(finalContent.trim());

console.log('=====
====');
        console.log(`[INFO] File Size    : ${fileStats.size} bytes`);
        console.log(`[INFO] Last Modified : ${fileStats.mtime.toLocaleString()}`);
        console.log(`[INFO] Absolute Path : ${filePath}`);

console.log('=====
====\n');
        } catch (error) {
            console.error('\n[!] An error occurred during file operations:', error.message);
        } finally {
            rl.close();
            console.log('>> Session closed. File operations completed successfully.\n');
        }
    }
}
// Execute Application
runFileManager();

```

7. CODE EXPLANATION

Code	Explanation
require('fs')	Imports Node.js core File System module to perform file I/O operations.
require('readline')	Imports core module providing a readable stream interface for terminal prompts.
require('path')	Resolves cross-platform file system paths across Windows and POSIX operating systems.
readline.createInterface()	Initializes a communication stream linking terminal input (process.stdin) and output (process.stdout).

<code>new Promise((resolve) => ...)</code>	Wraps callback-based <code>rl.question</code> into a Promise enabling readable sequential <code>async/await</code> syntax.
<code>fs.writeFileSync(path, data, 'utf-8')</code>	Synchronously creates and writes content to disk, overwriting any previous content if the file existed.
<code>fs.readFileSync(path, 'utf-8')</code>	Reads the file synchronously and decodes the binary byte buffer into a readable UTF-8 string.
<code>fs.appendFileSync(path, data, 'utf-8')</code>	Appends text to the end of the specified file without truncating or removing existing content.
<code>fs.statSync(path)</code>	Returns a <code>fs.Stats</code> instance providing file size in bytes, inode, timestamps, and block size.
<code>try { ... } catch (err) { ... }</code>	Traps file system runtime exceptions (such as locked files or disk errors) to prevent process crashes.
<code>rl.close()</code>	Closes input stream listeners and releases the terminal process back to the shell.

8. TEST CASES AND EXECUTION DATA

The following test cases were recorded after executing the Node.js application in the terminal.

Test Case	TC-01
Test / Input	Prompt Input: Filename <code>student_log.txt</code> and initial content <code>`"Student: N. Samhavaya</code>
Expected Result	File <code>student_log.txt</code> is created on disk and initial string is written successfully.
Actual Result	Terminal logged: [✓] File ' <code>student_log.txt</code> ' created and written successfully!.
Status	Pass

TC-01 Output:

```

PS C:\Users\COMPUTER\Desktop\expense_tracker\t2> node script.js

=====
NODE.JS FILE SYSTEM MANAGEMENT APPLICATION
=====

[1] Enter the filename to create (e.g., student_record.txt): student_log.txt

[2] Enter initial content to write to the file:
> Student : N. Samhavaya | Roll No : 99 | Branch : CSM

--> Creating and writing to "student_log.txt"...
[✓] File "student_log.txt" created and written successfully!

```

Test Case	TC-02
Test / Input	Read operation immediately following Step 1 write
Expected Result	Terminal outputs the exact string written to <code>student_log.txt</code> .
Actual Result	FILE CONTENTS (AFTER INITIAL WRITE) displayed the complete student record string accurately.

Status	Pass
--------	------

TC-02 Output:

```
-----
FILE CONTENTS (AFTER INITIAL WRITE):
Student : N. Samhavya | Roll No : 99 | Branch : CSM
-----
```

Test Case	TC-03
Test / Input	Append Operation & Final Content Display: Input "Enrolled Course: Full Stack Web Development (23CM4121)"
Expected Result	Content is appended without erasing initial data, and final content shows 154 bytes.
Actual Result	Appended content displayed alongside initial line; file size reported accurately as 154 bytes.
Status	Pass

TC-03 Output:

```
FINAL FILE CONTENTS (student_log.txt):
=====
Student : N. Samhavya | Roll No : 99 | Branch : CSM
Student : B. Prasuna | Roll No : 100 | Branch : CSM
=====
[INFO] File Size      : 104 bytes
[INFO] Last Modified  : 19/9/2026, 9:10:19 pm
[INFO] Absolute Path  : C:\Users\COMPUTER\Desktop\expense_tracker\t2\
student_log.txt
=====
```

9. OUTPUT

Output 1: Node.js Terminal File Management Execution

```
Problems Output Debug Console Terminal Ports
● PS C:\Users\COMPUTER\Desktop\expense_tracker\t2> node script.js

=====
NODE.JS FILE SYSTEM MANAGEMENT APPLICATION
=====

[1] Enter the filename to create (e.g., student_record.txt): student_log.txt

[2] Enter initial content to write to the file:
> Student : N. Samhavya | Roll No : 99 | Branch : CSM

--> Creating and writing to "student_log.txt"...
[✓] File "student_log.txt" created and written successfully!

--> Reading contents from file...
-----
FILE CONTENTS (AFTER INITIAL WRITE):
Student : N. Samhavya | Roll No : 99 | Branch : CSM
-----

[3] Enter additional content to append to the file:
> Student : B. Prasuna | Roll No : 100 | Branch : CSM

--> Appending content to "student_log.txt"...
[✓] Additional content appended successfully!

--> Reading final updated file contents...
=====
FINAL FILE CONTENTS (student_log.txt):
=====
Student : N. Samhavya | Roll No : 99 | Branch : CSM
Student : B. Prasuna | Roll No : 100 | Branch : CSM
=====
[INFO] File Size      : 104 bytes
[INFO] Last Modified  : 19/9/2026, 9:10:19 pm
[INFO] Absolute Path  : C:\Users\COMPUTER\Desktop\expense_tracker\t2\
student_log.txt
=====
```

Figure 1: Terminal Execution Output of Node.js File Manager

Supporting Files Used

The following files are used with this task:

- fileManager.js — Core Node.js script implementing sequential file operations.
- student_log.txt — Created data file containing written and appended student records.

10. RESULT

Thus, the **Node.js File Management Application** was successfully developed and executed. The application accepted user inputs through the readline interface, created and wrote data to a file, read and displayed the initial contents, appended additional content, and verified the updated final contents using the fs module.